PROGRAM-3

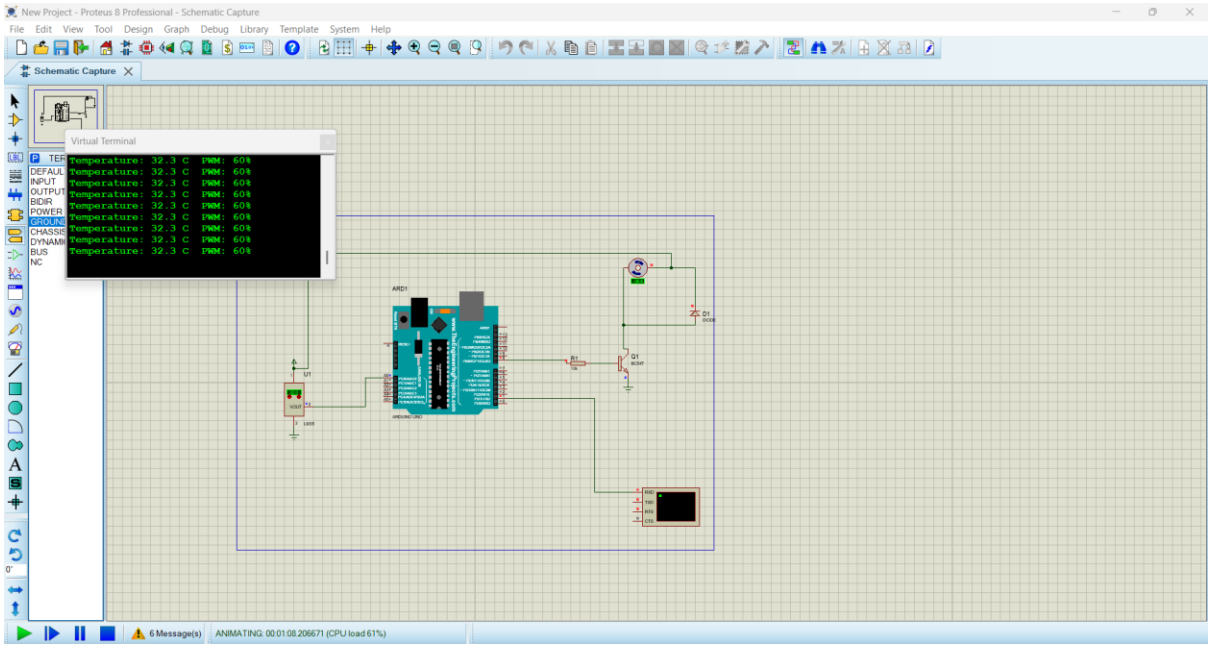
INDUSTRIAL HVAC CONTROL FIRMWARE: ADC TEMPERATURE ACQUISITION AND TIMER-INTERRUPT PWM FAN CONTROL

SOURCE CODE:

```
#define TEMP A0
#define FAN  8
volatile byte pwm = 0;
volatile byte count = 0;
ISR(TIMER1_COMPA_vect)
{
  count++;
  if (count >= 100)
    count = 0;
  digitalWrite(FAN, count < pwm);
}
void setup()
{
  Serial.begin(9600);
  pinMode(FAN, OUTPUT);
  TCCR1A = 0;
  TCCR1B = (1 << WGM12) | (1 << CS11);
  OCR1A = 199;
  TIMSK1 = (1 << OCIE1A);
  sei();
}
void loop()
{
  int adc = analogRead(TEMP);
  float temp = adc * 500.0 / 1023.0;
  if (temp < 25)
    pwm = 0;
  else if (temp < 30)
```

```
  pwm = 30;
 else if (temp < 35)
  pwm = 60;
 else
  pwm = 90;
 Serial.print("Temperature: ");
 Serial.print(temp, 1);
 Serial.print(" C  PWM: ");
 Serial.print(pwm);
 Serial.println("%");
 delay(1000);
}
```

OUTPUT:



OBSERVATION:

- **ADC Temperature Acquisition:** The Arduino UNO successfully acquired the analog output voltage from the LM35 temperature sensor connected to analog input pin `A0` and converted it into a digital value via its 10-bit ADC.
- **Temperature Conversion Accuracy:** The digital ADC reading was accurately converted into degrees Celsius using the formula $\text{temp} = \text{adc} \times 500.0 / 1023.0$, matching the LM35 sensitivity of $10\text{mV/}^\circ\text{C}$.

- **Fan OFF Condition (< 25°C):** When the simulated temperature dropped below 25°C, the system successfully set the PWM duty cycle to `0%`, keeping the HVAC DC motor completely turned OFF.
- **Low Fan Speed Condition (25–29°C):** For temperatures ranging from 25°C to below 30°C, the firmware correctly assigned a `30%` PWM duty cycle, resulting in slow/low fan operation.
- **Medium Fan Speed Condition (30–34°C):** When the temperature increased to the 30°C to 34°C range, the PWM duty cycle automatically stepped up to `60%` for medium fan cooling.
- **High Fan Speed Condition (≥ 35°C):** For high temperatures of 35°C and above, the system scaled the PWM duty cycle to its maximum threshold of `90%` for high-speed fan operation.
- **Timer1 Interrupt Software PWM:** Timer1 configured in CTC mode successfully generated periodic interrupts, driving software-based PWM on digital pin `D8` with a 100-step counter (`0` to `99`).
- **Transistor Driver Response:** The PWM signal from pin `D8` was successfully isolated and amplified through a $1\text{ k}\Omega$ base resistor into the BC547 NPN transistor to control the regional ground current of the DC motor.
- **Inductive Load Protection:** The 1N4007 flyback diode connected in reverse-biased configuration across the DC motor successfully suppressed voltage spikes and protected the circuit from inductive kickback.
- **Serial Monitoring & Virtual Terminal Output:** The system continuously transmitted real-time status updates via the serial port at 9600 baud, successfully displaying the exact measured temperature (in $^\circ\text{C}$) and corresponding PWM percentage on the Proteus Virtual Terminal.